JChatMind 面试介绍稿

整理时间：2026-03-25

一、1分钟项目介绍（精简版）

这个项目叫 **JChatMind**，是一个可落地的 AI Agent 系统，不是简单聊天框。技术栈是 **Spring Boot + Spring AI + MyBatis + PostgreSQL/pgvector + React + SSE。**

我主要做了四个核心能力：

- **Agent Loop（Think-Execute）**：用户提问后，Agent 会在“思考-执行工具-再思考”中循环，支持多步任务并限制最大步数防止死循环。
- **工具体系**：工具分固定和可选，按 Agent 配置动态装配，支持知识检索、数据库查询、邮件发送等能力。
- **RAG 知识库**：文档上传后自动解析分块、生成向量并入库 pgvector，查询时召回相关片段作为回答依据。
- **实时可视化**：通过 SSE 把 Agent 的执行状态和生成内容实时推送到前端，用户能看到完整推理执行过程。

我负责后端架构、Agent 主循环、工具调度、RAG 链路和前后端实时通信的打通。

二、3分钟深挖版（可直接背）

我这个项目叫 **JChatMind**，定位是可工程化落地的 AI Agent，而不是单轮 ChatBot。整体是前后端分离架构：后端用 Spring Boot + Spring AI + MyBatis + PostgreSQL/pgvector，前端用 React + Ant Design，实时通信用 SSE。

1）先讲完整链路

用户在前端发起问题后，后端先把消息落库，然后发布聊天事件，异步创建 Agent 运行实例。Agent 按照我实现的 Think-Execute 循环执行：先让模型判断下一步是否需要工具；需要则执行工具并把结果写回对话上下文；再进入下一轮思考，直到任务完成或触发最大步数保护。

2）Agent 运行时框架

我把 Agent 的配置做成可持久化模型：包括模型类型、可用工具、可访问知识库、消息窗口长度。运行时通过 Factory 动态装配：加载历史消息、处理不完整 tool-call 链、绑定工具回调、按模型路由 ChatClient，然后生成独立 Agent 实例执行。这让系统具备可配置、可扩展能力。

3）工具系统治理

工具分固定工具和可选工具。固定工具保证基础执行能力，可选工具由 Agent 配置控制启用。比如数据库工具限制为只读查询，避免模型误写数据。并且我关闭了框架默认自动工具执行，改成手动调度，目的是把权限、异常和审计都掌握在业务层。

4）RAG 全链路

文档上传后会自动走解析与分块、embedding 生成、向量入库、相似度检索。向量数据和业务数据统一在 PostgreSQL 体系内，部署成本低且一致性更好。Agent 在需要知识补充时会先检索，再基于召回上下文回答，降低幻觉。

5）实时体验与可解释性

通过 SSE 维护会话级连接，把 AI 生成内容和状态事件（规划/执行/完成）实时推送前端。用户能看到系统“正在做什么”，交互体验和可解释性明显优于黑盒式等待。

6）我的贡献与项目价值

我主导了后端架构和关键链路实现：Agent Loop、工具调度、RAG 入库检索、SSE 推送和前后端联调。项目价值在于把“调用模型 API”升级成了“可规划、可调用工具、可接知识库、可观察执行过程”的工程化 Agent 系统。

三、常见追问速答（加分）

Q：为什么用 SSE 而不是 WebSocket?
A：场景是服务端单向推送状态与内容，SSE 更轻量，接入和运维成本更低。

Q：如何防止 Agent 无限循环?
A：设置最大步数、终止工具、状态机与异常兜底，达到阈值即强制结束。

Q：如何控制工具调用风险?
A：工具分级治理 + 可选工具白名单 + 数据库只读限制 + 手动工具调度。